

ESC112 Tutorial 6

1 Introduction

topics to be covered:

- structures
- recursive structures – linked list, in particular

2 structures

- structures are basically compound data types created by programmers. they can have member variables of all kinds: int, float, char *, ..., even structures themselves
- once defined, they behave like usual variables. members can be accessed by dot. their address can be taken using &. pointers to them and arrays of them can be defined.
- two structures of the same type can be assigned to one another. this means in particular, they can be passed to/from functions.

Some examples are as follows.

2.1 example 1

```
1 struct date {
2     int day;
3     int month;
4     int year ;
5 };
6 struct account {
7     int acct_no;
8     char acct_type;
9     char name[80];
10    float balance;
11    struct date opening_date;
12 };
13
14 #include <stdio.h>
15 int main() {
16     struct account customer = {1234567890, 'S', "Shyam_Sharma", 25008.50,
17     {1, 1, 1990}};
18     printf("Account_Name:_%s\n", customer.name);
19     printf("Balance:_%%.2f\n", customer.balance);
20     printf("Opening_Date:_%d-%d-%d\n",
21           customer.opening_date.day,
22           customer.opening_date.month,
23           customer.opening_date.year);
24     struct account customer2 = customer;
25 }
```

remarks:

- initialization syntax is a hierarchy of braces
- access is hierarchy of dots

2.2 example 2

```
1 struct point origin, *pp = &origin;
2 printf("origin_is_(%d,%d)\n", (*pp).x, (*pp).y);
3 printf("origin_is_(%d,%d)\n", pp->x, pp->y);
4 struct point *ptarray[100];
```

- $(*p).x$ is equivalent to $p->x$

Exercise: take a student csv data with roll number and scores. sort them in two ways first by roll number and then by scores and print the two sorted lists. using structures avoids us parallel arrays. we still need to write two (quick) sort functions.

3 recursive structures

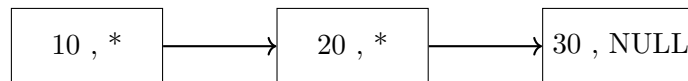
- a structure can't have a member of type itself but it can member pointer to itself. no problem as pointers are basically addresses of some fixed size.

- using this one can define recursive structures. we will see linked list today.

A linked list is a structure which has some data and also a pointer to the location of the next data, and so on. the last data has a null pointer denoting termination.

```
1 struct node {
2     int data;
3     struct node *next;
4 };
```

Items will be linked via pointers – need not be memory-contiguous. unlike arrays. advantage: flexible attachment/detachment.



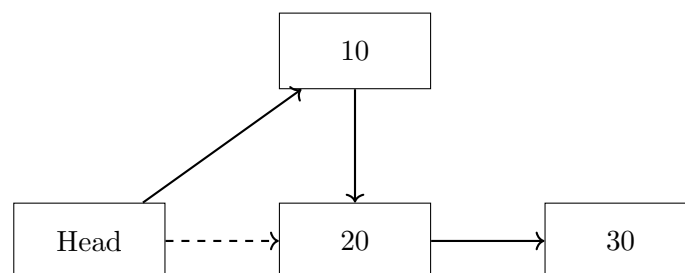
its use inside main function can be as follows.

```
1 struct node *new_node;
2 new_node = malloc(sizeof(struct node));
3
4 new_node->data = 10;
5 new_node->next = NULL;
```

note we are indirectly declaring one node by declaring a pointer to it and then allocating the pointer.

it creates a singleton list having value 10. head of the linked list must be separately maintained to access it. head is initially NULL. then items could be inserted in front.

```
1 struct node *head = NULL;
```



```
1 new_node->next = head;
2 head = new_node;
```

items can also be inserted in the end one by one as follows.

```

1 struct node *temp = head;
2
3 while (temp->next != NULL)
4     temp = temp->next;
5
6 temp->next = new_node;

```

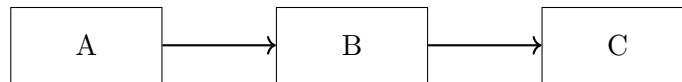
we can traverse the entire list starting from the head.

```

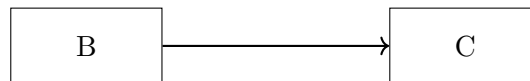
1 struct node *temp = head;
2
3 while (temp != NULL) {
4     printf("%d_", temp->data);
5     temp = temp->next;
6 }

```

deleting head node.



Remove A



```

1 struct node *temp = head;
2 head = head->next;
3 free(temp);

```

we need to always be careful of proper use of pointers, malloc, free while dealing with linked lists. deleting an intermediate node is left as an exercise.

we can search for a key as follows. similar to traversing.

```

1 struct node *temp = head;
2
3 while (temp != NULL) {
4     if (temp->data == key)
5         printf("Found\n");
6     temp = temp->next;
7 }

```

one wide use of linked list is for managing queues. here is such a problem.

4 Application: queue management

```
1 Problem: Multi-Class Queue Management System (Linked List in C)
2
3 You are required to implement a queue management system for the entry gate
  of a cricket match stadium using linked lists in C.
4
5 There are 5 ticket classes, numbered from 1 to 5:
6 - Class 1 has the highest priority and costs the most.
7 - Class 5 has the lowest priority and costs the least.
8
9 Each class maintains its own queue of fans. A fan can join only one class
  queue at a time.
10
11 Each fan is identified by a unique **alphanumeric ID** (consisting of
   letters and digits only, no spaces).
12
13 Your program must process a sequence of commands and simulate the system
   accordingly.
14
15 -----
16
17 Input Format
18
19 The input consists of multiple lines. Each line is one of the following
   commands:
20
21 1. Q ID priority_no
22    - A fan with unique alphanumeric identifier ID joins the queue of class
      priority_no.
23    - priority_no is an integer from 1 to 5.
24    - The fan is added to the end of the corresponding class queue.
25
26 2. W ID
27    - Search for the fan with identifier ID in all 5 queues.
28    - If found, remove the fan from the queue they are currently in.
29    - If not found, do nothing.
30
31 3. E
32    - The gate opens for one fan.
33    - The fan at the front of the smallest-numbered non-empty queue (
      highest priority) is allowed entry.
34    - That fan is removed from the system.
35
36 -----
```

```

1 Output Requirements
2
3 - For every E operation:
4   - Print the ID of the fan who gets entry.
5   - If all queues are empty, print:
6     EMPTY
7
8 - For every W ID operation:
9   - If the fan is found and removed, print:
10    REMOVED ID
11   - If the fan is not found, print:
12    NOT FOUND
13
14 - No output is required for Q operations.
15
16 -----
17
18 Final Output (after processing all commands)
19
20 After all commands are processed, print:
21
22 1. The list of all fans who entered the stadium, in the exact order they
23    were dequeued:
24    ENTERED: ID1 ID2 ID3 ...
25
26 2. The final state of all queues from Class 1 to Class 5 in the following
27    format:
28
29    Class 1: ID ID ID
30    Class 2: ID ID ID
31    Class 3: ID ID ID
32    Class 4: ID ID ID
33    Class 5: ID ID ID
34
35    - If a class queue is empty, print:
36      Class k: EMPTY
37
38 -----
39
40 Constraints
41
42 - Maximum number of fans in the system: 100
43 - IDs are alphanumeric strings (letters and digits only, no spaces)
44 - Maximum length of an ID: 20 characters
45 - IDs are unique at any point in time
46 - Total number of operations <= 1000

```

```

1 -----
2
3 Sample Input
4
5 Q A101 3
6 Q VIP9 1
7 Q B77 2
8 E
9 W B77
10 E
11 E
12 E
13
14 -----
15
16 Sample Output
17
18 VIP9
19 REMOVED B77
20 A101
21 EMPTY
22 EMPTY
23 ENTERED: VIP9 A101
24 CLASS 1: EMPTY
25 CLASS 2: EMPTY
26 CLASS 3: EMPTY
27 CLASS 4: EMPTY
28 CLASS 5: EMPTY
29
30 -----
31
32 Implementation Requirements
33
34 - Use linked lists to implement the queues.
35 - Maintain 5 separate queues (one for each class).
36 - Ensure efficient insertion at the end and deletion from the front.
37 - The W ID operation requires searching across all queues.

```

Let us discuss the solution.

4.1 Header and initialize_list functions

"typedef" is a new thing for us. it simply renames a data type to another. In the following, we can now write simply "Fan" to mean "struct fan" and "List" to mean "Fan *" i.e. "struct fan *".

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #define MAXCLASSES 5
6 #define LINEMAX 100
7 #define MAXFANS 100
8
9 struct fan{
10     char *ID;
11     int priority; //1 to 5
12     struct fan *next;
13 };
14
15 typedef struct fan Fan;
16 typedef Fan * List;
17
18 void initialize_list(List entry[], int n){
19     for(int i = 0; i < n; i++){
20         entry[i] = NULL;
21     }
22 }

```


4.2 main function

```
1
2
3 int main(){
4     //array for head of queues
5     List entry[MAXCLASSES + 1]; //0 not used for easy indexing
6
7     initialize_list(entry, MAXCLASSES + 1);
8
9     //array for entered IDs
10    char *entered[MAXFANS];
11
12    char line[LINEMAX], *tok_ID, *tok_priority;
13    List f;
14    int n = 0;
15    while(fgets(line, LINEMAX, stdin) != NULL){
16        switch(line[0]){
17            case 'Q':
18                //get id
19                tok_ID = strtok(line + 2, "_");
20                tok_priority = strtok(NULL, "\n");
21                enqueue(entry, tok_ID, atoi(tok_priority));
22                break;
23            case 'E':
24                f = dequeue(entry, MAXCLASSES + 1);
25                if(f != NULL){
26                    printf("%s\n", f->ID);
27                    n = add_entered(entered, n, f->ID, MAXFANS);
28                    free_listnode(f);
29                }
30                else
31                    printf("EMPTY\n");
32                break;
33            case 'W':
34                tok_ID = strtok(line + 2, "\n");
35                withdraw_queue(entry, MAXCLASSES + 1, tok_ID);
36                break;
37            default:
38                printf("invalid_command\n");
39                return 1;
40        }
41    }
42
43    //print entered list
44    print_entered(entered, n);
45    free_entered(entered, n);
46
47    //print waiting list
48    print_waiting(entry, MAXCLASSES + 1);
49    free_entry(entry, MAXCLASSES + 1);
50    return 0;
51 }
```

reads each line and tokenizes to get the command: queue/enter/withdraw; and calls a suitable function. array entry holds the head of each class list.

4.3 enqueue function

```
1 void enqueue(List entry[], char *ID, int priority){
2     List l = entry[priority];
3     if(l == NULL){
4         l = (List) malloc(sizeof(Fan));
5         l->ID = strdup(ID);
6         l->priority = priority;
7         l->next = NULL;
8         entry[priority] = l;
9         return;
10    }
11
12    while(l->next != NULL){
13        l = l -> next;
14    }
15    l->next = (List) malloc(sizeof(Fan));
16    l->next->ID = strdup(ID);
17    l->next->priority = priority;
18    l->next->next = NULL;
19    return;
20 }
```

notice the two cases are differently handled. depending on if the list for the person's priority level is empty or not.

4.4 dequeue function

```
1 List dequeue(List entry[], int n){
2     //priority 1 first. priority 0 is always empty
3     List temp;
4     for(int i = 0; i < n; i++){
5         if(entry[i] != NULL){
6             temp = entry[i];
7             entry[i] = entry[i] -> next;
8             return temp;
9         }
10    }
11    return NULL;
12 }
```

notice how the higher priority list is dequeued earlier.

4.5 withdraw_queue function

```
1 void free_listnode(List l){
2     if(l == NULL)
3         return;
4
5     if(l->ID != NULL)
6         free(l->ID);
7     free(l);
8 }
9
10 void withdraw_queue(List entry[], int n, char *ID){
11     List l, prevl, temp;
12     for(int i = 0; i < n; i++){
13         l = entry[i];
14         prevl = NULL;
15         while(l != NULL){
16             if(strcmp(l->ID, ID) == 0){
17                 if(prevl == NULL){
18                     //head removes
19                     temp = entry[i];
20                     entry[i] = entry[i]->next;
21                     free_listnode(temp);
22                 }
23                 else{
24                     prevl->next = l->next;
25                     free_listnode(l);
26                 }
27                 printf("REMOVED_%s\n", ID);
28                 return;
29             }
30             prevl = l;
31             l = l->next;
32         }
33     }
34
35     //not found
36     printf("NOT_FOUND\n");
37     return;
38 }
```

ID was filled using strdup calls. so, while freeing, we free ID first, then the node itself.

4.6 other functions

we maintained a list of entered persons entered. it needs a deep copy – simple pointer copying will fail if the person is no more in queue and freed in main. we print this list in the end.

```

1 int add_entered(char *entered[], int n, char *ID, int max){
2     if(n == max)
3         return -1;
4
5     //n is the first empty index
6     //deep copy needed -- could be removed
7     entered[n] = strdup(ID);
8
9     return ++n;
10 }
11
12 void print_entered(char *entered[], int n){
13     printf("ENTERED:");
14     for(int i = 0; i < n; i++)
15         printf("_%s", entered[i]);
16     printf("\n");
17 }

```

printing the waiting queue is straightforward.

```

1 void print_waiting(List entry[], int n){
2     //ignore i = 0
3     List l;
4     for(int i = 1; i < n; i++){
5         printf("CLASS_%d:", i);
6         l = entry[i];
7         if(l == NULL){
8             printf("_EMPTY%s", (i == n - 1)? "" : "\n");
9             continue;
10        }
11
12        while(l != NULL){
13            printf("_%s", l-> ID);
14            l = l -> next;
15        }
16        printf("%s", (i == n - 1)? "" : "\n");
17    }
18 }

```

we free all the lists and the entered array as usual.

```

1 void free_entry(List entry[], int n){
2     List l, temp;
3     for(int i = 0; i < n; i++){
4         l = entry[i];
5         while(l != NULL){
6             temp = l;
7             l = l -> next;
8             free_listnode(temp);
9         }
10    }
11 }
12
13 void free_entered(char *entered[], int n){
14     for(int i = 0; i < n; i++)
15         free(entered[i]);
16 }

```